

Chrome DevTools Workshop

Master the Art of Debugging

Welcome!

What We'll Cover Today

1. **Elements Panel** - Inspect & modify HTML/CSS
2. **Console Panel** - Debug JavaScript & interact with the page
3. **Sources Panel** - Set breakpoints & step through code
4. **Network Panel** - Monitor API requests & performance
5. **Performance Panel** - Identify bottlenecks & optimize

Each section includes **hands-on exercises!**

Workshop Goals

By the end of this workshop, you'll be able to:

-  Inspect and modify any webpage in real-time
-  Debug JavaScript errors efficiently
-  Set breakpoints and step through code
-  Analyze network requests and responses
-  Identify and fix performance issues

Workshop Materials

Clone the repository:

```
git clone https://github.com/YourRepo/chrome-dev-tools-workshop.git
```

Structure

```
chrome-dev-tools-workshop/  
├── elements/      # HTML/CSS inspection  
├── console/       # JavaScript debugging  
├── source/        # Breakpoints & stepping  
├── network/       # API monitoring  
└── performance/  # Performance profiling
```

Opening DevTools

Action	Windows/Linux	Mac
Open DevTools	<code>F12</code> or <code>Ctrl+Shift+I</code>	<code>Cmd+Option+I</code>
Inspect Element	<code>Ctrl+Shift+C</code>	<code>Cmd+Shift+C</code>
Open Console	<code>Ctrl+Shift+J</code>	<code>Cmd+Option+J</code>
Command Menu	<code>Ctrl+Shift+P</code>	<code>Cmd+Shift+P</code>

Or: Right-click → "Inspect"

Elements Panel

Inspect & Modify HTML/CSS

Elements Panel Overview

What Can You Do?

-  Inspect any element on the page
-  Edit HTML content live
-  Modify CSS styles in real-time
-  Debug layout with the Box Model
-  Test responsive design

The Panel Layout

Left Side	Right Side
HTML DOM tree	Styles panel
Element hierarchy	Computed styles
Searchable (<code>Ctrl+F</code>)	Box model diagram

Selecting Elements

Method 1: Inspect Mode

Click the **inspect icon** → Hover over page → Click to select

Method 2: Right-Click

Right-click any element → **"Inspect"**

Method 3: Search

Press **Ctrl+F** / **Cmd+F** in Elements panel

Editing HTML

Double-click to Edit

```
<h1>Original Text</h1>  
<!-- Double-click the text to change it -->
```

Right-click Options

- **Edit as HTML** - Full HTML editing
- **Delete element** - Remove from DOM
- **Hide element** - Toggle visibility

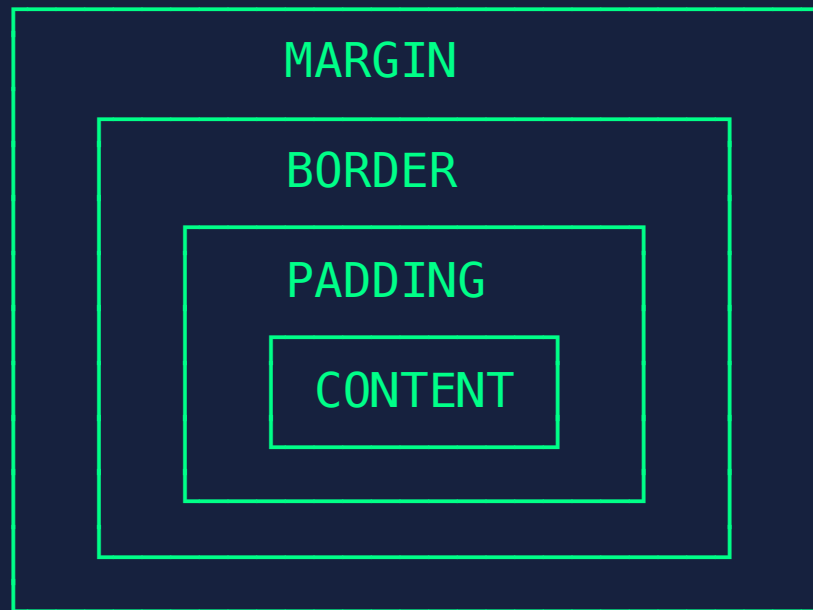
Editing CSS

```
.hero {  
  background-color: #007bff; /* Click to change! */  
  padding: 50px;  
}
```

Quick Actions

-  Click color square → Color picker
-  Click property name → Add new property
-  Uncheck checkbox → Disable property

The Box Model



Hover over each section to highlight it on the page!

Responsive Design Mode

Toggle: **Ctrl+Shift+M** / **Cmd+Shift+M**

-  Device presets - iPhone, iPad, Pixel, etc.
-  Custom dimensions - Set exact width/height
-  Orientation - Portrait/Landscape toggle

Exercise: Elements Panel

Open `elements/index.html`

1. Change hero background from **blue** to **green** (`#4CAF50`)
2. Fix the **misaligned feature boxes** (check margins!)
3. Add a new nav item: "**Blog**"
4. Test in **responsive mode**

Console Panel

Debug JavaScript & Interact with the Page

Console Panel Overview

-  View logs - `console.log()`, errors, warnings
-  Debug errors - Stack traces, error messages
-  Run JavaScript - Execute code live
-  Inspect objects - Expand and explore data

Console Output Types

```
console.log("Info message"); //  Default
console.warn("Warning message"); //  Yellow
console.error("Error message"); //  Red
console.table([{a: 1}, {a: 2}]); //  Table format
```

Filtering

Click filter buttons: **Errors** | **Warnings** | **Info**

Understanding Errors

✗ Uncaught TypeError: Cannot read property 'length' of undefined
 at processData (script.js:42)
 at handleClick (script.js:28)

Part	Meaning
Error type	<code>TypeError</code> - wrong data type
Message	What went wrong
Stack trace	Where it happened (click to jump!)

Running JavaScript in Console

```
// Select elements
document.querySelector('h1')

// Modify the page
document.body.style.backgroundColor = 'red'

// Copy to clipboard
copy(someObject)
```

Console Shortcuts

Variable	Description
<code>\$0</code>	Currently selected element
<code>\$_</code>	Result of last expression
<code>\$(sel)</code>	Shorthand for <code>querySelector</code>
<code>\$\$ (sel)</code>	Shorthand for <code>querySelectorAll</code>

```
$0.textContent = "Modified!"
```

Exercise: Console Panel

Open `console/index.html`

1. View the logged values - what is `number` ?
2. Find the calculation bug - sum should be 15!
3. Click "Start Async Task" - is error handled?
4. Run: `document.querySelector('h1').textContent = "Fixed!"`

Sources Panel

Breakpoints & Debugging

Sources Panel Overview

-  **Set breakpoints** - Pause code execution
-  **Step through code** - Line by line
-  **Inspect variables** - See values in Scope panel
-  **View call stack** - Trace function calls

Setting Breakpoints

Click the Line Number

```
function processUser(user) {  
  console.log(user.name); // ← Click line number  
  return user;  
}
```

A blue marker appears = breakpoint is set!

Types of Breakpoints

Type	How to Set
Line breakpoint	Click line number
Conditional	Right-click → "Add conditional breakpoint"
Logpoint	Right-click → "Add logpoint"

Stepping Through Code

Button	Shortcut	Action
 Resume	F8	Continue to next breakpoint
 Step Over	F10	Execute line, move to next
 Step Into	F11	Go inside function call
 Step Out	Shift+F11	Exit current function

The Scope Panel

When paused, see variable values:

```
▼ Local
  user: {id: 1, name: "Alice"}
  result: undefined
▼ Closure
  config: {debug: true}
```

If a variable isn't listed, it's **not accessible** in current scope!

The Call Stack

Shows how you got here:

```
▼ Call Stack
  processUser      (script.js:42)
  handleClick      (script.js:28)
  (anonymous)      (script.js:15)
```

Click any frame to see that function's code and variables.

Exercise: Sources Panel

Open `source/index.html`

1. Set a breakpoint in `complexAsyncFunction`
2. Click "Trigger Debug" and step through
3. Find the Promise bug - it resolves AND rejects!
4. Find the scope bug - variable not accessible

Network Panel

Monitor Requests & Debug APIs

Network Panel Overview

-  **Monitor requests** - See all HTTP traffic
-  **Inspect headers** - Request & response details
-  **Analyze timing** - Find slow requests
-  **Throttle network** - Simulate slow connections

Reading the Network Panel

Column	Description
Name	URL/endpoint
Status	HTTP status code (200, 404...)
Type	xhr, fetch, document...
Time	Total request time
Waterfall	Visual timeline

Status Code Colors

-  Green = Success (2xx)
-  Red = Error (4xx, 5xx)
-  Yellow = Redirect (3xx)

Request Details

Click any request to see tabs:

Tab	Shows
Headers	Request/response headers
Preview	Formatted response
Response	Raw response body
Timing	Detailed timing breakdown

Understanding Timing

Queueing	■	Waiting for browser
DNS	■	Resolving domain
Connection	■	TCP handshake
Waiting	■	Server processing (TTFB)
Download	■	Receiving response

Long **TTFB**? → Server issue | Long **Download**? → Large response

Network Throttling

1. Click "No throttling" dropdown
2. Select: **Fast 3G** | **Slow 3G** | **Offline**

Why Throttle?

- Test loading states
- Find performance issues
- Ensure graceful degradation

Useful Features

-  **Preserve Log** - Keep requests after navigation
-  **Disable Cache** - Force fresh requests
-  **Copy as cURL** - Right-click → Copy as cURL
-  **Block Requests** - Right-click → Block URL

Exercise: Network Panel

Open `network/index.html`

1. Click "Fetch Posts" and watch the request
2. Inspect the response - how many posts?
3. Enable "Slow 3G" throttling
4. Check the Timing tab - what's the TTFB?

Performance Panel

Identify & Fix Bottlenecks

Performance Panel Overview

-  Record profiles - Capture execution timeline
-  Analyze flame chart - See what's running
-  Find bottlenecks - Identify slow code
-  Track memory - Detect leaks

Recording a Profile

1. Open **Performance** panel
2. Click **Record** button (●)
3. Interact with the page
4. Click **Stop**
5. Analyze the results!

The Flame Chart

Color	Meaning
 Yellow	JavaScript execution
 Purple	Layout/Rendering
 Green	Painting
 Gray	Idle/Other

What to Look For

- **Wide yellow blocks** = Long-running JavaScript
- **Many purple blocks** = Layout thrashing
- **Red corners** = Dropped frames (jank)

Problem: Blocking the Event Loop

```
// ❌ BAD: Blocks for seconds
for (let i = 0; i < 500000000; i++) { }

// ✅ GOOD: Break into chunks
function processChunk() {
  // Process small batch
  if (moreWork) setTimeout(processChunk, 0);
}
```

Problem: Layout Thrashing

```
// ❌ BAD: 100,000 reflows!  
for (let i = 0; i < 100000; i++) {  
    container.appendChild(element);  
}  
  
// ✅ GOOD: One reflow  
const fragment = document.createDocumentFragment();  
// ... add to fragment ...  
container.appendChild(fragment);
```



Exercise: Performance Panel

Open `performance/index.html`

1. Before "Start Test": Try hovering & typing
2. Record a profile while clicking "Start Test"
3. Find the blocking code (huge yellow block!)
4. Identify the DOM thrashing

Advanced Exercises

Level Up Your Skills

Advanced Topics Available

Section	Advanced Topics
Elements	Grid/Flexbox debugging, Animations
Console	Live Expressions, <code>monitor()</code>
Sources	Conditional breakpoints, Logpoints
Network	WebSocket debugging, Local Overrides
Performance	Memory leaks, Heap Snapshots

Each section has an `advanced/` folder!

Key Takeaways

Elements Panel

Inspect & modify HTML/CSS in real-time

Console Panel

View logs, errors, use `$0` and `$$()` shortcuts

Sources Panel

Set breakpoints, step through code

Key Takeaways (cont.)

Network Panel

Monitor requests, use throttling, check Timing tab

Performance Panel

Record profiles, look for long yellow blocks



Resources

- [Chrome DevTools Docs](#)
- [DevTools Tips](#)
- [DevTools Challenges](#)



Shortcuts Reference

Action	Windows/Linux	Mac
Open DevTools	F12	Cmd+Option+I
Inspect Element	Ctrl+Shift+C	Cmd+Shift+C
Step Over	F10	F10
Step Into	F11	F11
Resume	F8	F8

 Questions?

Happy Debugging!  → 

Next Steps

1.  Complete the **basic exercises**
2.  Try the **advanced exercises**
3.  Practice on **your own projects**
4.  Explore: **Application, Memory, Lighthouse**

The best way to learn DevTools is to **use them every day!**